An Executable Exhaustive Catalogue for Conjecture 00000001029

idealistichacker (AI-assisted)

October 29, 2026

Abstract

For every prime power q and all natural n, k, a single terminating algorithm constructs a finite field of order q from operation tables and returns a duplicate-free list of labelled $S(2, k, n)_q$ designs. Membership is equivalent to the actual subspace-design condition, and every design over any field of order q occurs after a fixed field isomorphism. This proves the source's finite-explicit-list assertion, in particular when $k \mid n$. It is exhaustive enumeration, not an efficient algorithm, a nonemptiness theorem, or an isomorphism classification.

1 Statement and Meaning of the Output

Both source languages request a finite explicit list of $S(2, k, n)_q$ for each n and $k \mid n$. Here q has its standard finite-field meaning: $q = p^m$ for a prime p and $m > 0$. Let K be a field of order q and $V = K^n$. A design is a set $\mathcal{D}$ of linear subspaces of V such that

$$2 \le k \le n, \qquad \forall W \in \mathcal{D},\ \dim_K W = k, \qquad \forall U \le V,\ \dim_K U = 2 \Longrightarrow \exists! W \in \mathcal{D},\ U \subseteq W. \tag{D}$$

The ambient space is finite, so every set of its subspaces is finite. This is proved rather than assumed for arbitrary block families.

Theorem 1. *There is an executable function $A(q, n, k)$, with no input field or oracle. If q is a prime power, it returns a field table T on q labels and a finite duplicate-free list L of families of vector-carrier codes. For every field K of order q, fix a field isomorphism from the field described by T to K, applied in every coordinate. Then an arbitrary set $\mathcal{D}$ of subspaces of K^n satisfies* (D) *if and only if exactly one member of L decodes and transports to $\mathcal{D}$. If $2 \le k \le n$ fails, L is empty.*

The theorem holds for all n, k, so its $k \mid n$ specialization gives the requested assertion. No quotient by linear, semilinear or incidence isomorphism is taken: distinct labelled designs remain distinct. The statement contains no bound on running time or output size. The theorem supplies the specified finite computation and proves its exact output criterion; it does not replace computation by classical selection of an unknown list.

2 Constructing the Field

Enumerate $z, o \in \{0, \dots, q-1\}$ and all pairs of binary operation tables a, b on these labels, in the finite tuple order used by `allFunctions`. Retain the first table satisfying:

- $z \neq o$; addition is associative, $z + a = a$, and each element has a left additive inverse;
- multiplication is associative and commutative, $oa = a$, and multiplication distributes over addition;
- for each $a \neq z$ there exists b with $ab = o$.

All tests are finite. These are sufficient minimal field axioms, not merely necessary tests: Mathlib's `Field.ofMinimalAxioms` constructs an actual `Field` from them. Additive negatives and nonzero inverses are defined by the least successful bounded search; the inverse of zero is explicitly zero. `TableElem` is a fresh wrapper for the labels, avoiding the unrelated modular operations on Lean's `Fin q`. Its cardinality is proved to be q.

Lemma 1. *For every prime power q, this table search succeeds. It uses only q, and its returned operations define a computable field of cardinality q.*

Proof. Write $q = p^m$ with p prime and $m > 0$. Mathlib constructs a finite field $\mathrm{GF}(p, m)$ with exactly p^m elements. Relabel its elements by $\{0, \ldots, q-1\}$. The transported zero, one, addition and multiplication form one of the enumerated tables and pass every test. Thus a first successful table exists. The search and inverse searches are executable finite operations. The possibly noncomputable Galois-field construction and its relabelling occur only in this existence proof, not in the algorithm's dependency path. □

For arbitrary natural q the algorithm is total: it returns failure if no valid table is found. No field-existence assertion is made for $q = 0$ or $q = 1$.

3 Actual Subspaces and the Incidence Filter

Work now over the selected field K_0 of order q. Enumerate the finite vector set $X = K_0^n$. Among all subsets $S \subseteq X$, retain those containing zero and closed under addition and multiplication by every scalar. Such an S decodes to the actual submodule $\{x \in V : x \in S\}$. Conversely, the carrier of every actual submodule is one of these codes.

Lemma 2. *Encoding and decoding give inverse bijections between closed carrier codes and actual subspaces. They preserve equality and containment. For a closed carrier code S and its subspace W,*

$$|S| = |W| = q^{\dim W}.$$

Consequently, $|S| = q^d$ if and only if $\dim W = d$.

Proof. The zero, addition and scalar closure tests are exactly the submodule axioms, so decoding gives a submodule and encoding returns its original carrier. The two inverse identities imply equality and containment correspondence. The carrier and the subtype representing its subspace are in bijection. The finite-vector-space cardinality theorem gives $|W| = q^{\dim W}$. Since every field has $q \geq 2$, the function $d \mapsto q^d$ is injective on natural numbers. □

Let E_d consist of the closed carrier codes of cardinality q^d. When $2 \leq k \leq n$, enumerate all $B \subseteq E_k$ and retain precisely those satisfying

$$\forall U \in E_2, \quad \big|\{W \in B : U \subseteq W\}\big| = 1. \tag{I}$$

Outside this parameter guard return the empty list. The guard occurs in both the running algorithm and the actual target (D). There is no exceptional vacuous design for $(n, k) = (0, 0)$.

Lemma 3. *Every retained B decodes to a design satisfying* (D)*. Every actual design has exactly one retained labelled code family.*

Proof. By Lemma 2, E_k is exactly the set of k-subspaces in carrier form, and E_2 is exactly the set of 2-subspaces in that form. Containment of codes is actual subspace containment. Cardinality one in (I) is equivalent to existence and uniqueness of the containing block, proving soundness. For completeness, encode each block of an actual design; the same equivalences establish (I). Applying the two inverse encoding identities to finite families proves uniqueness. For an arbitrary set of blocks, inject its subspaces into the finite set of vector subsets; this proves finiteness and justifies the finite-family representation without an extra hypothesis. □

4 Executable Lists and Change of Field

The returned list is not formed by a noncomputable selection from a finite set. Enumerate coordinate tuples with `allFunctions`. Enumerate their sublists, convert each to a finite carrier set, remove duplicates, and apply the closure and cardinality tests to obtain a list of E_k. Enumerate its sublists in the same way to get all block families, filter by (I), and remove duplicates. For any finite set S, it occurs among the sublist-to-set codes of a list x if and only if all elements of S occur in x: the reverse direction uses the sublist obtained by filtering x by S. This proves that the executable list has exactly the finite-set catalogue's membership relation. Explicit duplicate removal proves the returned list is duplicate-free.

Now let K be any field with q elements. Finite fields of the same cardinality are isomorphic; Mathlib's `FiniteField.ringEquivOfCardEq` supplies such an isomorphism $e : K_0 \simeq K$ for the semantic proof. Applied coordinatewise, it is a semilinear equivalence $K_0^n \simeq K^n$. Its induced subspace order isomorphism preserves containment and carrier cardinality. Lemma 2 and injectivity of powers of q show that it also preserves dimension. Transporting finite block families is a bijection, hence preserves the unique-containing-block condition in both directions. Together with Lemma 3, this proves Theorem 1 for every field model. Neither K nor e is an input to the running catalogue.

5 Formalization and Verification Scope

The Lean source uses Lean 4.33.1 and Mathlib commit `0df444a360eaa60ab8c11dca51a86af692955474`. The main all-model conclusion is `QOnly.source_conjecture`; arbitrary set-valued block families are covered by `SetFamilies.source_conjecture_sets`. `QOnly.catalogue` is the executable entry point. Source definitions, dimension/cardinality bridges, field construction, guarded incidence filtering, explicit lists, and model transport are separate modules. Every advertised theorem is accompanied by actual type and axiom output. Only the standard logical axioms `propext`, `Classical.choice`, and `Quot.sound` are permitted; no extra axioms or proof placeholders are used.

Strict local evaluation of the actual q-only entry returns one family containing one block for $(q, n, k) = (2, 2, 2)$, the expected whole space design. The invalid input $(2, 0, 0)$ returns an empty catalogue; $q = 0, 1$ return no field model. These are small runtime diagnostics, not substitutes for the general proofs. A prior combined diagnostic including $n = 3$ exceeded its 180-second budget; no successful $n = 3$ evaluation is claimed, and that failure is retained. No large parameter catalogue is claimed to have been computed.

The source is conjecture `00000001029`, blob `001aa6f79c0aaaa70d0907eb42b9fb5169beb631`, checked in both languages. This manuscript establishes a literal exhaustive-list result. It makes no claim of structural classification, a list with one representative per isomorphism class, efficient search, nonexistence or existence of a design at any other parameter, first solve credit, or upstream acceptance.

References

[1] The Mathlib community, *Mathlib*, fixed revision cited above. Used results include `Field.ofMinimalAxioms`, `Module.card_eq_pow_finrank`, `GaloisField.card`, and `FiniteField.ringEquivOfCardEq`. All are imported from the fixed source tree; the submitted source records the exact interfaces.